Tiny-GEMM: Packed INT4 Triton GEMM for Decode-Heavy LLM Inference

Robert Zhang

Abstract

Small-batch LLM decoding is dominated by narrow GEMMs that stress memory bandwidth and launch overhead rather than peak FLOPs. We present Tiny-GEMM, a packed INT4 GEMM kernel in Triton targeted at decode-heavy shapes, and a measurement-driven analysis of when quantization helps or hurts. We compare FP16, dequantized FP16, and INT4 fused kernels, attribute performance using hardware counters, and isolate dequantization overhead with microbenchmarks. Results on an A10G show substantial gains for wide FFN decode shapes while highlighting regimes where dequant overhead dominates. We provide a reproducible evaluation harness and a focused set of figures that connect kernel behavior to systems-level inference constraints.

1 Introduction

LLM inference workloads differ sharply from training: decoding operates at batch sizes of 1–8 with skinny matrices and tight latency budgets. These shapes are often memory-bound, and naive quantization can underperform if dequantization overhead is not amortized. This work explores packed INT4 GEMM in Triton and provides a systems-oriented evaluation emphasizing utilization, bottleneck regimes, and deployment-relevant shapes.

Contributions.

- A decode-focused INT4 GEMM kernel with static configuration selection and measurement-driven tuning.
- A regime analysis of INT4 decode GEMMs that identifies when quantization helps or hurts, supported by FP16 and dequantized FP16 baselines.
- A causal attribution methodology combining hardware counters and microbenchmarks to explain performance transitions.

2 Background and Motivation

Decode GEMMs (e.g., Q/K/V projections and FFN layers) typically use $M \in \{1, 2, 4, 8\}$ and hidden dimensions in the 4K–16K range. For these shapes, the dominant costs are memory movement and kernel launch overhead, not peak compute.

Table 1: Representative decode shapes (Llama-style).

Layer	M	K	N
Q/K/V proj	1–8	4096	4096
KV proj	1–8	4096	1024
FFN up	1–8	4096	14336
FFN down	1–8	14336	4096

This motivates INT4 weight packing and fused computation to reduce traffic and improve utilization.

3 Method

We implement packed INT4 GEMM in Triton with per-tensor quantization and bit-packed weight tensors. The kernel unpacks INT4 values into FP32 accumulators inside the kernel and uses static configurations keyed by shape family and batch bucket. We evaluate three baselines:

- **FP16**: `torch.matmul` as the standard deployment baseline.
- **Dequantized FP16**: INT4 quantize $\rightarrow$ dequant $\rightarrow$ FP16 GEMM.
- **INT4 fused**: packed INT4 GEMM kernel (Tiny-GEMM).

4 Experimental Setup

All experiments run on an NVIDIA A10G GPU. We focus on decode shapes derived from Llama-style models: Q/FFN projections with K, N in $\{4096, 14336\}$ and KV projections with $N = 1024$. Profiling uses Nsight Compute; wall-clock timings are reported from the benchmark harness (profilers replay kernels and inflate timings).

5 Results

5.1 Story Figure

We summarize the core claim with a single panel that connects performance regimes, utilization shift, and dequantization cost.

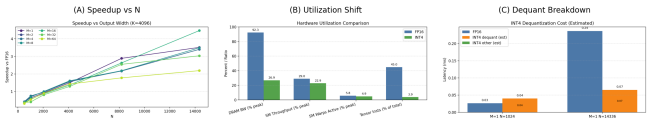

Figure 1: INT4 helps when bandwidth dominates and dequant is amortized.

5.2 Where Does INT4 Help?

We identify the regime where INT4 improves decode performance. Figure 2 shows speedup vs FP16 as N increases (fixed $K = 4096$). INT4 underperforms on narrow projections but provides strong gains for wide FFN layers. Figure 3 groups results by layer family, showing the same trend across M=1 and M=8.

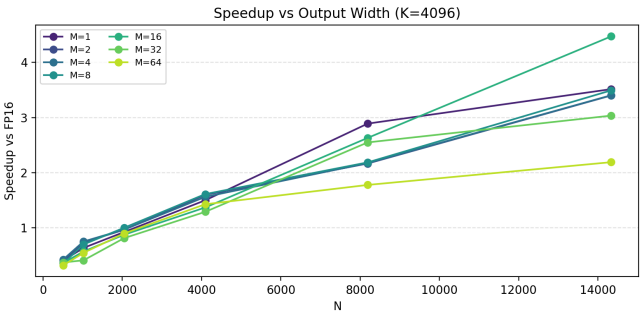

Figure 2: Speedup vs output width N (decode shapes, $K = 4096$).

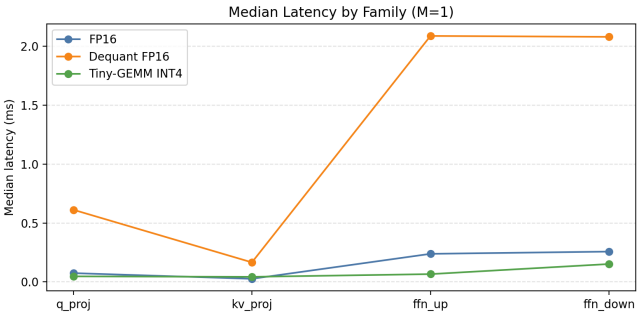

Figure 3: Latency by shape family (M=1).

5.3 Prefill vs Decode

We compare prefill-like ($M \gg 1$) and decode-like ($M \leq 8$) regimes using a fixed shape family grouping. INT4 benefits are stronger in decode for wide FFN layers but remain mixed for narrow projections.

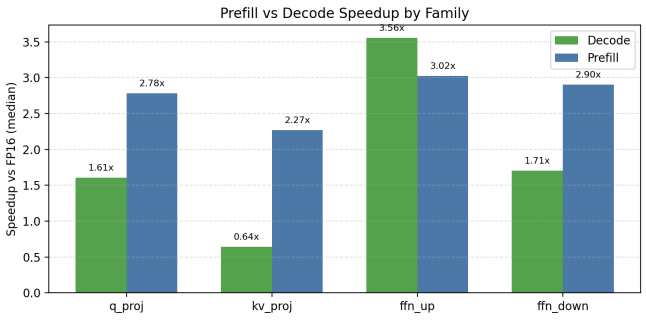

Figure 4: Prefill vs decode speedup by family.

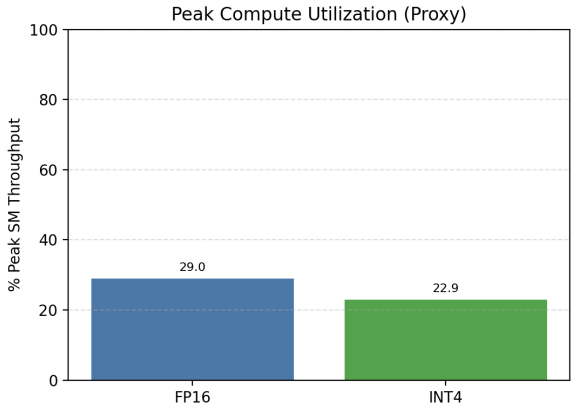

Figure 5: Peak compute utilization proxy (SM throughput) for FP16 vs INT4.

5.4 Why Does INT4 Help (or Hurt)?

We isolate dequantization overhead and attribute bottleneck shifts. The dequantization microbenchmark (Figure 6) shows that fixed INT4 overhead dominates narrow projections, explaining regressions on KV-like shapes. Hardware counters provide a complementary view: INT4 shifts utilization compared to FP16 (Figure 7), while a roofline scatter illustrates the memory-bound regime (Figure 8).

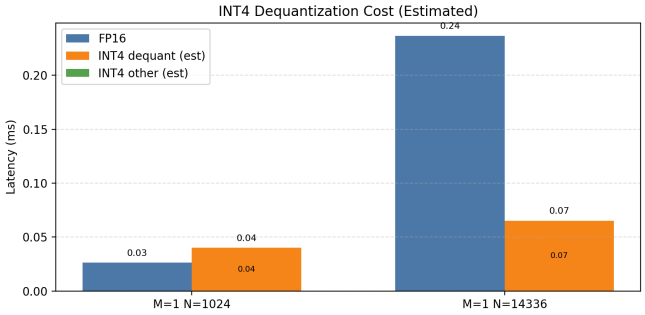

Figure 6: Dequantization breakdown for representative decode shapes.

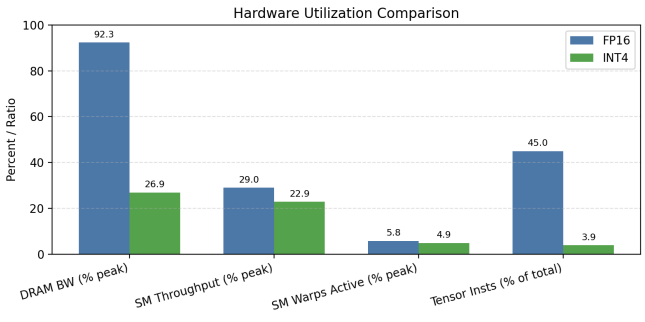

Figure 7: Hardware utilization comparison (FP16 vs INT4).

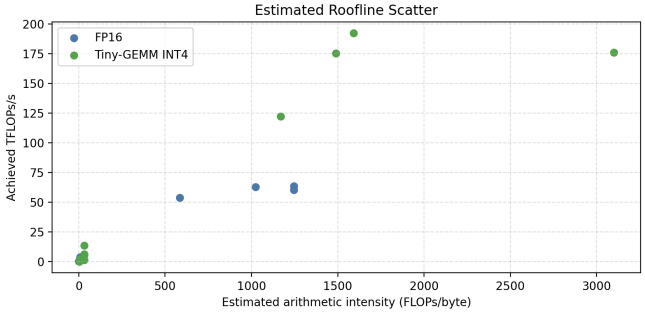

Figure 8: Estimated roofline scatter (arithmetic intensity vs TFLOPs).

5.5 A Regime Model for INT4 Decode GEMMs

We model total latency as:

$$T_{\text{total}} = T_{\text{launch}} + T_{\text{mem}}(W) + T_{\text{dequant}} + T_{\text{compute}}. \quad (1)$$

Narrow projections are dominated by fixed T_{dequant} and limited parallelism, while wide FFN layers are dominated by T_{mem}. INT4 reduces T_{mem}, creating a transition boundary in arithmetic intensity space. Figure 9 visualizes this boundary.

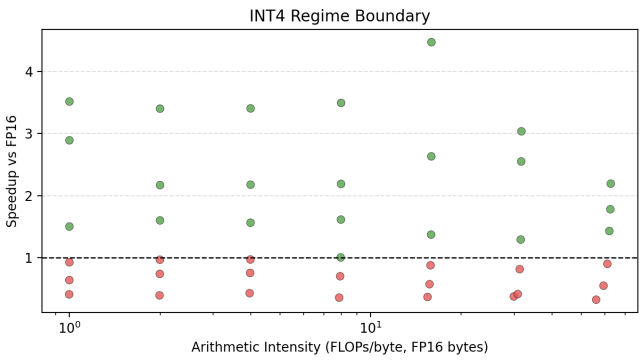

Figure 9: Regime boundary: speedup vs arithmetic intensity.

5.6 When Does INT4 Help in Systems Terms?

We examine batch scaling and throughput to connect kernel behavior to decode latency constraints. Figure 10 shows latency vs batch size for a representative decode shape; Figure 11 reports tokens/sec.

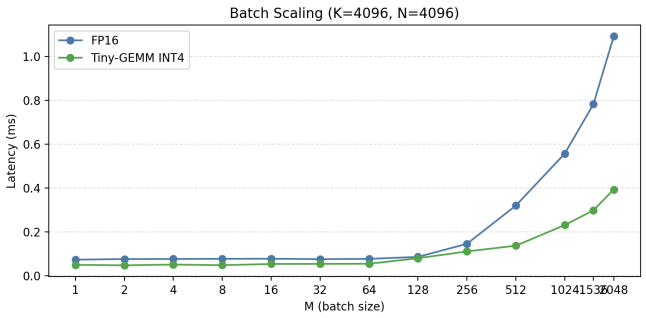

Figure 10: Batch scaling for decode ($K = N = 4096$).

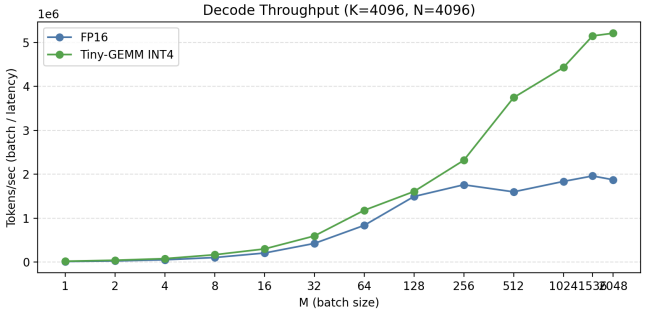

Figure 11: Decode throughput (tokens/sec) vs batch size.

5.7 Regime Map and Kernel Composition

To summarize regimes, we include a decode heatmap (Figure 12) and kernel time composition (Figure 13).

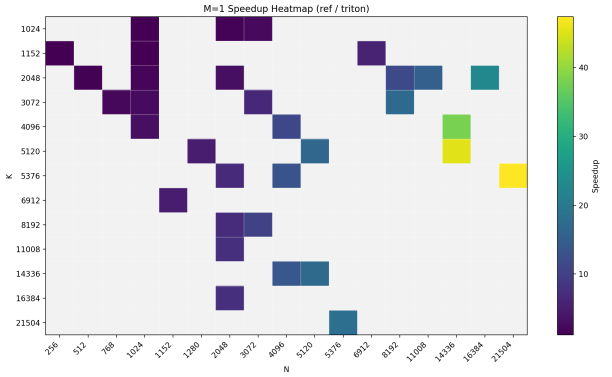

Figure 12: Decode regime map (M=1) across K and N.

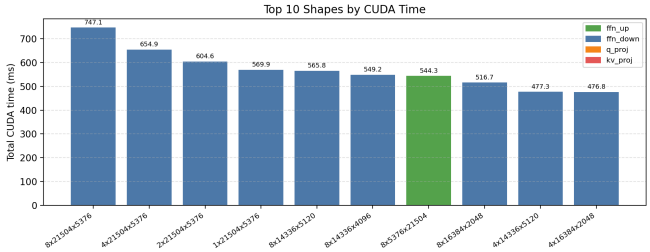

Figure 13: Top shapes by CUDA time (Nsight Systems).

6 Discussion

The results support a clear regime interpretation: INT4 helps when arithmetic intensity is sufficient to amortize dequantization and when memory traffic is a dominant bottleneck. For narrow projections (e.g., KV), fixed overheads and limited parallelism can outweigh the bandwidth savings. These findings align with practical inference constraints in decode-heavy serving. Our results suggest that weight-only INT4 should not be applied universally in decode pipelines; kernel-aware deployment decisions are necessary to avoid regressions on narrow projections.

7 Related Work

Quantization for inference has been explored extensively in SmoothQuant, GPTQ, and AWQ, among others, focusing on accuracy-preserving weight-only and activation-aware techniques. Systems efforts such as FlashInfer and TensorRT-LLM emphasize kernel specialization and end-to-end inference throughput. CUTLASS and cuBLASLt provide highly optimized GEMM kernels that serve as deployment baselines. Tiny-GEMM complements these lines of work by focusing on the decode regime and providing a measurement-based model of when weight-only INT4 improves latency.

8 Limitations

This study targets a single GPU and a fixed set of decode shapes. The kernel is not yet optimized for all regimes (e.g., split-K for $M = 1$), and the INT4 kernel does not currently exploit tensor core INT4 MMA pipelines. Future work should extend to additional hardware (L4/A100), incorporate split-K or persistent kernel strategies, and explore INT8/FP8 comparisons.

9 Conclusion

Tiny-GEMM demonstrates that packed INT4 kernels can significantly improve decode performance for wide FFN shapes while exposing the boundaries where dequant overhead dominates. By pairing multi-baseline benchmarks with hardware counters and targeted microbenchmarks, we provide a systems-level narrative of when and why INT4 helps in real inference settings.